

Week 7 — Block headers, UTXO validation, micropayments, and payment channels

1) Blockchain block header

The first major slide explains the **Bitcoin block header**. The lecture says each block has a header of **80 bytes**, and that this header contains **6 fields**:

1. Version
2. Previous block hash
3. Merkle root hash
4. Timestamp
5. Target hash / difficulty target
6. Nonce

It also gives the block header formula and says the block header hash is computed using **double SHA-256**.

This is very important because the block header is like the **identity card** of the block. The block may contain many transactions, but the header is the compact structured summary that miners hash during mining and that nodes use to link blocks together.

Let's understand each field very clearly.

1.1 Version

The version field tells the network which rule set or protocol signaling is being used. You can think of it like the block saying: "I was created under this version of the protocol." That matters when upgrades happen and nodes need to understand what kind of block they are receiving.

1.2 Previous block hash

This is what links one block to the earlier block. It is the field that makes the blocks form a chain. If block **b** contains the hash of block **b-1**, then block **b** is explicitly tied to that parent block. This is one of the reasons tampering is hard: changing an earlier block changes its hash, which breaks the later links.

1.3 Merkle root hash

This is the summary of all the transactions inside the block. Instead of putting all transactions into the header, the block computes a **Merkle root**, which acts like one compact fingerprint for the whole transaction set. If any transaction changes, the Merkle root changes. So this field helps maintain transaction integrity.

1.4 Timestamp

This records the approximate block creation time. The lecture says up to about **2 hours of variation is acceptable**, because Bitcoin nodes are not obsessed with perfect clock synchronization. That is an important detail: Bitcoin uses time, but not with exact real-time precision like a bank database might.

1.5 Target hash / difficulty target

This field defines how hard it is to find a valid block header hash. The lecture says it adjusts every **2016 blocks**, which is about **two weeks**, to try to keep average block generation around **10 minutes**. This is how Bitcoin keeps mining difficulty adaptive.

1.6 Nonce

This is the field miners keep changing when trying to find a valid block header hash. The nonce is basically the miner's adjustable trial value during Proof of Work.

1.7 Block header hash

The lecture gives the formula:

Block_Header_Hash = SHA256(SHA256(Block_Header)).

That means Bitcoin takes the header, hashes it once, then hashes the result again. This double hashing is part of Bitcoin's design.

1.8 Big picture

The block header is what miners are really competing over. They are not hashing the entire giant block every time in the mining loop. They hash the compact header, whose fields summarize the block and link it into the chain.

2) Can we pay for coffee in BTC?

This slide is one of the most important "motivation" slides in the week. It asks:

- Bitcoin price is too high for a single cup of coffee

- average block generation time is 10 minutes, do we want to wait that long?
- do we want every coffee purchase broadcast to the whole world?
- billions of coffee cups are consumed every day, can Bitcoin process all these transactions?

This slide is basically setting up the problem that the rest of the week solves.

2.1 Price issue

At first glance, people say: “BTC is expensive, how can you buy coffee with it?” But this is partly a mental-unit problem. Later the lecture will say: stop thinking in full bitcoins, think in **satoshis**. So the “price too high” issue is more psychological than mathematical.

2.2 Waiting 10 minutes

This is a serious practical issue. Bitcoin’s average block time is about 10 minutes. If every small retail payment had to wait for on-chain confirmation, that would be terrible user experience. Nobody wants to stand at a coffee shop counter for 10 minutes, or longer if multiple confirmations are needed.

2.3 Privacy issue

If every tiny purchase is written on the public blockchain, then your daily life becomes publicly analyzable. That is not great for privacy. Buying coffee is a small everyday act, but if every such act is permanently recorded on a public ledger, it becomes a surveillance-style issue.

2.4 Throughput issue

The lecture asks whether Bitcoin can handle billions of coffee transactions per day. The answer is basically no, not directly on-chain. So the lecture is pushing you toward the idea that **on-chain Bitcoin alone is not ideal for micropayments**.

That is why the payment-channel idea becomes so important later.

3) Bitcoin statistics revisited

The lecture repeats the Bitcoin stats:

- 19.98 million bitcoins in circulation
- 21 million total max
- blockchain size 715 GB
- 144 blocks/day

- 450 new BTC/day
- around 7 transactions per second
- price around \$66k

The key question on the slide is:

Does Bitcoin look like a peer-to-peer electronic cash system?

This is a very meaningful question.

3.1 Why ask this?

Because Satoshi's original vision described Bitcoin as a **peer-to-peer electronic cash system**.

But if Bitcoin is:

- slow for small payments,
- limited in TPS,
- expensive relative to micropayment use,
- and very public,

then we have to ask whether the current base layer alone fulfills that original cash-like vision.

3.2 The deeper message

Sir is not saying Bitcoin failed completely. He is saying:

the base layer gives decentralization and security, but maybe it needs **additional mechanisms** to behave like everyday payment money.

That is exactly what payment channels and Lightning-style thinking are about.

4) Bitcoin as decentralized digital currency

The lecture reminds us that Satoshi envisioned Bitcoin as a peer-to-peer digital currency for all kinds of online payments.

This is important because it keeps the original goal in mind:

Bitcoin was not meant to be only a “digital gold” store of value. It was also imagined as money for payment.

So the lecture is asking:

how can we move from “Bitcoin is secure but slow on-chain” to “Bitcoin can actually support everyday payments”?

That is the whole arc of Week 7.

5) Bitcoin as a micropayment platform

The lecture asks:

Is there a way to make Bitcoin a micropayment platform without sacrificing decentralization or security?

That sentence is basically the central question of the entire week.

5.1 Why “without sacrificing” matters

Many solutions could make payments faster by becoming centralized. For example, one company could keep an internal database of small BTC-like balances. But then you lose the trustless, decentralized nature of Bitcoin.

So the challenge is:

- keep Bitcoin’s security
- keep Bitcoin’s decentralization
- but make small payments fast and cheap

That is exactly why payment channels are introduced.

6) Bitcoin scripts revisited — what opcodes can do

The lecture now revisits Bitcoin Script and lists what opcodes can do:

- check signatures with `OP_CHECKSIG`
- combine conditions with `OP_IF`, `OP_ELSE`, `OP_ENDIF`
- require multiple approvals with `OP_CHECKMULTISIG`
- lock funds by time with `OP_CHECKLOCKTIMEVERIFY`
- lock funds by delay with `OP_CHECKSEQUENCEVERIFY`

This is very important, because it is showing that Bitcoin Script is more expressive than just “normal payment.”

6.1 Signature checking

This is the classic ownership check: prove that the spender has the right private key.

6.2 Combined conditions

Bitcoin Script can encode logic like:

- Alice OR Bob can spend
- if condition A then path X else path Y

So transactions can have branches.

6.3 Multiple approvals

With multisig, you can require:

- 2 of 3 people
- 3 of 5 people
- etc.

This is useful for shared custody and advanced protocols.

6.4 Time locks

You can prevent spending until a certain time or block height.

6.5 Delay locks

You can require waiting some amount of time after a transaction is created/confirmed.

This is especially important in payment channels and punishment mechanisms later.

The main takeaway is:

Bitcoin Script can enforce much more than simple one-key ownership.

7) What is possible with Bitcoin Script?

The next slide organizes script possibilities into:

- key & script locks: P2PKH, P2MS, P2SH, P2WPKH, P2WSH, P2TR
- time locks: absolute and relative
- hash locks
- HTLCs (Hashed Time Lock Contracts)

Then the slide says:

scripts are low-level and hard to write and Bitcoin Script is stack-based and not human-friendly.

This slide is basically expanding the design space.

7.1 Key/script locks

These are different ways of saying who can spend and how.

7.2 Time locks

These let funds be restricted until a certain time or after a certain delay.

7.3 Hash locks

These let spending depend on knowledge of a secret whose hash is known.

7.4 HTLC

This combines hash locks and time locks and becomes essential for routed payments later in Lightning-style systems.

7.5 Why “hard to write” matters

Bitcoin Script is powerful, but not developer-friendly. It is low-level and stack-based, so it is easy to make mistakes.

That is why the lecture introduces **Miniscript** next.

8) Miniscript

The lecture says:

- Miniscript is like TypeScript for Bitcoin
- just as TypeScript turns into JavaScript for browsers,
- Miniscript turns into Bitcoin Script for the blockchain
- it lets developers build “smart contracts” on Bitcoin without needing a PhD in opcodes.

This is a very nice analogy.

8.1 Why TypeScript analogy?

Because TypeScript is easier for humans to reason about, while JavaScript is what the browser ultimately runs.

Similarly:

- Miniscript = more human-readable policy layer
- Bitcoin Script = low-level executable form

8.2 Why does this matter?

Because serious Bitcoin applications need expressive scripts, but raw Script is too error-prone.

So Miniscript helps developers describe spending policies in a more structured way.

8.3 Example mental model

Instead of manually writing painful stack-based script logic, you can express a policy like:

- Alice can spend now
- OR Bob can spend after 30 days
- OR Alice and Bob together can spend anytime

Then Miniscript can compile that into proper Bitcoin Script.

That is the core idea.

9) Raw Bitcoin Script vs Miniscript

The lecture explicitly compares them:

Raw Bitcoin Script

- hard to read
- easy to mess up
- not intuitive
- hard to verify

Miniscript

- structured
- readable
- matches the policy
- easier to reason about

This is likely a short-answer question.

Main point:

Miniscript improves human usability and reasoning, while raw Script remains low-level and difficult.

10) Bitcoin transactions — UTXO model revisited

Now the lecture comes back to UTXO basics but explains them in a more practical way.

It says:

- Bitcoin transactions use the UTXO model
- a UTXO transaction has one or more inputs and one or more outputs
- each output has value and scriptPubKey
- each input references exactly one past output
- input contains scriptSig in legacy
- scriptSig is empty in SegWit and witness carries unlocking data
- $\text{sum}(\text{inputs})$ must be at least $\text{sum}(\text{outputs})$
- Bitcoin therefore does not require user accounts.

This is a very strong summary slide.

10.1 Outputs define locked value

Every output says:

- how much value
- under what locking condition

10.2 Inputs reference earlier outputs

An input does not point to an account balance.

It points to a specific earlier output.

10.3 No user-account table needed

Because ownership is tracked through spendable outputs, not through account rows.

That is one of the defining design choices of Bitcoin.

11) Spending an output

The next slide uses the parent-child transaction language:

- parent transaction = funding transaction
- child transaction = spending transaction
- value is locked in parent output's scriptPubKey
- child input identifies parent output and provides scriptSig
- scriptSig + scriptPubKey execute on stack
- if result is TRUE, output is unlocked and fully spent by child outputs.

This is one of the cleanest explanations of Bitcoin spending.

11.1 Parent output is the thing being spent

The earlier transaction created a spendable output.

11.2 Child input points to it

The new transaction says:

"I want to spend that exact output."

11.3 Unlocking + locking

The child proves it can unlock the parent output, then creates new locked outputs for future spending.

So Bitcoin is really a chain of locking and unlocking events.

12) Transaction chaining

The lecture says Bitcoin transactions form a chain too: each transaction spends outputs of earlier transactions and creates new outputs for future use. A transaction can also have multiple inputs.

This is a beautiful conceptual point.

People usually think only **blocks** form a chain. But **transactions also form a graph/chain of spending relationships**.

So there are two linked structures:

- block chain

- transaction spending chain

That is very important conceptually.

13) UTXO database

The lecture says:

- UTXOs are scattered across blocks
- nodes maintain a UTXO database, called `chainstate.db` in Bitcoin Core
- purpose is to speed up validation
- validation algorithm:
 1. for each input, if parent UTXO not in DB → error
 2. if signature doesn't match owner → error
 3. if total input value < total output value → error
 4. if successful, remove parent UTXOs and add child UTXOs.

This is an excellent practical slide.

13.1 Why keep a UTXO database?

Because otherwise nodes would have to search all history every time they validate a transaction.

That would be very inefficient.

13.2 What does the DB contain?

Only currently **unspent** outputs.

Not all historical outputs — just the still-spendable ones.

13.3 Validation algorithm in plain words

To validate a new transaction:

- check the inputs exist and are unspent
- check spender is authorized
- check you are not creating more value than you consume
- if valid, mark old inputs as spent and add the new outputs as spendable

This is exactly how the system state moves forward.

Lecture 14

14) Which child transactions are valid?

The lecture shows a parent transaction with several outputs and asks which child transactions are valid if signatures match and outputs are less than the referenced UTXO value. It says both shown child transactions are valid because they point to different parent output indices, **Idx 1** and **Idx 2**.

This is a subtle but important point.

14.1 Why can both be valid?

Because they do **not** compete for the same parent output.

They reference different outputs of the same parent transaction.

So even though they come from the same parent transaction ID, they are spending different pieces of value.

14.2 Important lesson

A transaction ID alone is not enough to identify the spent coin.
You also need the **index**.

That is why Bitcoin inputs use:

- previous txid
- output index

Together those identify the exact referenced UTXO.

15) Race between transactions

The next slide shows two child transactions trying to spend the **same** parent output **Idx: 2**, and says:

- the one that gets confirmed wins the race

- maybe it is the one created earlier?
- maybe the one with higher fee?
- this is double spending
- the two child transactions are in a race.

This is very important.

15.1 Why is this a race?

Because only one of the two can win.

If both child transactions reference the same UTXO, then they are trying to spend the same value twice.

That is the UTXO version of double spending.

15.2 Which one wins?

The one the network confirms first into the accepted chain.

Often higher fees help a transaction get mined sooner, but the lecture intentionally leaves it as a race concept rather than a fixed law.

15.3 Main lesson

In Bitcoin, double spending often appears as:
two transactions competing for the same UTXO.

That is a very strong exam sentence.

16) Making payments — valid but wasteful transaction

The lecture shows Alice owning 1.5 BTC in one output and wanting to pay 0.5 BTC to Bob. It asks whether a child transaction creating only a 0.5 BTC output to Bob is valid, and says yes — it is valid because Alice owns a valid UTXO and $\text{sum}(\text{inputs}) > \text{sum}(\text{outputs})$. But the remaining 1 BTC becomes miner fee, so Alice would lose it.

This is a really nice teaching example.

16.1 Why is it valid?

Because the rules only require:

- referenced input exists and is yours
- total output does not exceed input total

So technically it passes.

16.2 Why is it bad?

Because any leftover value not assigned to an output becomes transaction fee.

So if Alice spends 1.5 BTC input and creates only 0.5 BTC output, then she effectively pays 1 BTC fee.

16.3 Lesson

A transaction can be valid but economically foolish.

That is why change outputs matter.

17) Payment with a change output

The next slide fixes the problem. It says Alice should pay herself the remaining BTC through a **second output**, which is like receiving change. This creates:

- one output to Bob (0.5 BTC)
- one output back to Alice (1 BTC) as change.

This is the standard Bitcoin payment pattern.

17.1 Why is it called change?

Because it is just like getting change from a larger cash note.

17.2 Why is this necessary?

Because Bitcoin UTXOs are consumed whole. If the input UTXO is larger than the intended payment, you must explicitly create a new output returning the remainder.

17.3 Main takeaway

Bitcoin payments often have **two outputs**:

- recipient output
- sender change output

That is a very common transaction shape.

18) Collaborative transaction

This is another big concept.

The lecture says:

- Alice and Bob agree on inputs and outputs
- one creates an unsigned transaction
- unsigned transaction is shared
- Alice signs all inputs and outputs with her private key
- Bob signs all inputs and outputs with his private key
- they exchange signatures
- one holding all signatures has the fully signed transaction
- one holding only some signatures has a **PSBT** (Partially Signed Bitcoin Transaction)
- only fully signed transaction can be broadcast.

This is very important because payment channels later depend on collaborative transaction creation.

18.1 What is collaborative here?

Both parties agree on the whole transaction structure before broadcasting.

18.2 Why partially signed?

Because one party may sign first, then send it to the other, who adds their signature later.

18.3 What is PSBT?

A partially signed bitcoin transaction is basically an unfinished-but-progressing transaction package.

It is very important in wallets, multisig, and channel-style setups.

18.4 Main takeaway

Not every Bitcoin transaction is created by one person alone. Some transactions are jointly constructed and jointly signed.

19) BTC price vs a cup of coffee

The lecture then returns to the coffee-payment problem and visually says:
Bitcoin seems better suited for buying the entire coffee shop than a single cup.

This is partly humorous, but it reinforces the practical concern:
on-chain Bitcoin feels too heavy for tiny daily purchases.

20) Start thinking in satoshis

The lecture says:

- 1 BTC = 10^8 sats
- assume BTC price is \$100,000
- then 1 sat $\approx$ \$0.001
- one cup of coffee $\approx$ \$1 $\approx$ 1000 sats.

This is trying to fix the psychological barrier.

20.1 Why think in sats?

Because saying “0.00001000 BTC” feels awkward and expensive-looking.

But saying “1000 sats” feels much more like a usable everyday currency unit.

20.2 Why this matters

For micropayments, human-friendly denomination matters.

So the lecture is saying:
Bitcoin may still feel usable if you think in the smaller unit.

20.3 But is that enough?

Not yet — because fees are still a problem.

That is the next slide.

21) Transaction fee problem

The lecture says:

- one cup of coffee $\approx$ 800 to 1000 sats
- average transaction fee $\approx$ 400 to 800 sats
- so it is still difficult to use BTC for micropayments.

This is the real bottleneck.

21.1 Why is this bad?

If your fee is close to the payment amount, the system is impractical for small retail transactions.

21.2 Example

Paying 1000 sats for coffee with a 600-sat fee is absurd in everyday use.

So even if denomination is solved, fee economics still make on-chain micropayments weak.

22) Transaction fee depends on size, not value

The lecture explains:

- Bitcoin fee does not depend on transaction value
- it depends on transaction size in bytes
- a \$1 payment can pay same fee as a \$100,000 transfer if byte size is the same.

This is one of the most important Week 7 insights.

22.1 Why is this true?

Miners care about block space, not the dollar value of the transfer.

A block has limited capacity. So what matters is how many bytes your transaction occupies.

22.2 Why is this a micropayment problem?

Because small-value payments do not get “small-value fees” automatically.

If the byte size is similar, the fee is similar.

22.3 Why is this also an opportunity?

Because if you can avoid putting every small payment on-chain, you can save a lot.

That is the transition to off-chain commitment updates.

23) Reducing the fee for micropayments

The lecture gives the key idea:

- make one high-value **funding transaction** on-chain
- pay one fee for it
- then make many micropayments by updating balances **off-chain** using commitment transactions
- finally settle the latest state on-chain with another fee
- so instead of paying many on-chain fees, you pay only for opening and closing.

This is the core payment-channel idea.

23.1 Funding transaction

This is the initial on-chain transaction that locks some value into a shared setup.

23.2 Off-chain commitment transactions

These represent updated balances after each micropayment but do not normally go on-chain immediately.

23.3 Final settlement

Only the last agreed state is eventually published on-chain when closing.

23.4 Fee savings

If you make many coffee purchases inside one channel, you avoid paying full on-chain fee every single time.

This is the practical economic motivation for payment channels.

24) Funding and commitment transactions

The lecture then defines the two key terms.

Funding transaction

- a 2-of-2 multisig transaction on blockchain
- locks funds in shared ownership

Commitment transaction

- returns the funds of a two-party channel to their owners according to current balance split.

This is extremely important.

24.1 Why 2-of-2 multisig?

Because both channel participants jointly control the funding output.

Neither should be able to spend it alone without the channel rules.

24.2 What does commitment transaction do?

It represents the current “who owns how much” inside the channel.

So if Alice has spent some sats to Bob, the commitment transaction reflects the updated split.

24.3 Why obtain a pre-signed spending transaction first?

The lecture says before publishing the funding transaction, the funder first obtains a pre-signed recovery/spending transaction from the counterparty so funds can always be recovered even if cooperation stops.

That is a major safety rule:

never lock money on-chain unless you already have a way to get it back if the other party disappears.

25) Unobserved events

The lecture gives a Monday/Tuesday/Wednesday coffee example:

- Monday: Alice buys 2 coffees → balances 98,000 / 2,000
- Tuesday: buys 3 coffees → balances 95,000 / 5,000
- Wednesday: buys 5 coffees → balances 90,000 / 10,000
- only the final state is settled on-chain
- intermediate states never went on-chain and remain unobserved.

This is a beautiful example.

25.1 Why “unobserved events”?

Because the blockchain never sees each intermediate coffee purchase.

It only sees:

- channel opening
- final closing state

25.2 Privacy advantage

This is good for privacy because not every coffee purchase gets broadcast publicly.

25.3 Efficiency advantage

It is also good for cost and scalability because many updates happen without consuming block space.

This is one of the biggest practical benefits of payment channels.

26) Payment channel — the definition

The lecture defines a payment channel as a setup that allows two parties to make payments **without contacting the blockchain** for every payment. They can update balances as frequently as they want, and only when they close the channel does the final transaction go on-chain.

This is the central concept of Lecture 14.

26.1 What is the essence?

A payment channel is like opening a private mini-ledger between two parties, backed by blockchain security.

26.2 What remains on-chain?

- channel opening
- channel closing

26.3 What stays off-chain?

- the repeated intermediate payment updates

26.4 Why is it challenging?

The lecture says making the payment channel secure and trustless is a major challenge.

That means:

how do you stop cheating if updates are not immediately on-chain?

That challenge becomes the basis of later lectures, but even in Week 7, sir wants you to understand the motivation.

27) Full payment-channel flow

The final slide gives the flow:

1. Alice and Bob decide to open a payment channel
2. Alice funds it with 100,000 sats
3. They create a 2-of-2 multisig funding tx
4. Bob signs a commitment transaction to pay Alice back
5. Once Alice has this safe commitment, funding tx is broadcast
6. Alice buys coffee and they update balances
7. New commitment transaction is created
8. Alice buys more coffee and they update again
9. This continues
10. They eventually close the channel.

This is the whole Week 7 conclusion.

27.1 Why sign the commitment before broadcasting funding?

For safety. Alice should never broadcast a funding transaction unless she already knows she can recover her funds.

27.2 Why repeated commitment updates?

Because each new commitment reflects the newest state of payment distribution.

27.3 Why only final state on-chain?

To save fees and support many micropayments efficiently.

Week 7 — Full connected story

Now let me connect Week 7 as one big story.

Lecture 13 begins by explaining the Bitcoin block header: a compact 80-byte structure containing version, previous block hash, Merkle root, timestamp, difficulty target, and nonce. This header is what miners hash, and it links blocks together while summarizing the transaction set through the Merkle root. From there, the lecture asks a practical question: can Bitcoin really be used to buy something as small as a cup of coffee? Bitcoin may be decentralized and secure, but on-chain payments are slow, public, and limited in throughput. With roughly 10-minute block times and only about 7 transactions per second, it is hard to imagine the base layer alone handling billions of small daily purchases.

The lecture then revisits Bitcoin Script and shows that it can do much more than plain signature checks. With opcodes like `OP_CHECKSIG`, `OP_CHECKMULTISIG`, `CLTV`, and `CSV`, Bitcoin can enforce conditional spending rules, multisig approvals, time locks, and delay locks. It also introduces Miniscript as a more readable way to express these policies before compiling them down into raw Bitcoin Script. Then the lecture returns to the UTXO model and explains it in practical validation terms: each input references exactly one previous output, each output defines a value and a locking script, and nodes keep a UTXO database so they do not have to scan the full blockchain every time they validate a new transaction.

Lecture 14 then shows how UTXO spending behaves in practice. Two transactions can both be valid if they point to different output indices of the same parent transaction, but if two child transactions try to spend the same UTXO, they are in a race and only one can win. The lecture also gives the important example that a Bitcoin payment can be technically valid yet economically silly: if Alice spends a 1.5 BTC output to send Bob only 0.5 BTC and forgets to create a change output, the leftover 1 BTC becomes fee. That is why standard Bitcoin payments usually have both a payment output and a change output. The lecture also introduces collaborative transactions and PSBTs, where multiple parties jointly construct and sign a transaction before it can be broadcast.

Finally, the lecture returns to the coffee-payment problem and proposes the core solution: stop putting every tiny payment directly on-chain. Instead, use one on-chain funding transaction to lock value into a shared 2-of-2 multisig output, then update balances off-chain through commitment transactions after every purchase. Only the opening and final closing need to touch the blockchain. Intermediate coffee purchases remain off-chain, unobserved, cheaper, and more private. This structure is called a payment channel. It preserves Bitcoin's security anchor while making micropayments more practical by avoiding an on-chain fee for every small payment. So the entire logic of Week 7 is: Bitcoin's base layer is too heavy for everyday micropayments, but payment channels allow fast, cheap, and more private repeated payments without sacrificing the blockchain as the settlement foundation.

Most important Week 7 topics to memorize

These are the highest-priority ones from Lecture 13 and 14:

- block header fields
- block header hash
- why Bitcoin struggles for coffee / micropayments
- why TPS and 10-minute block time matter
- what Bitcoin Script can do
- Miniscript
- UTXO rules recap
- spending an output
- parent transaction vs child transaction
- transaction chaining
- UTXO database / chainstate
- simplified validation algorithm
- different output indices can both be valid
- race between transactions
- double spending as competition for same UTXO
- valid but wasteful transaction
- change output
- collaborative transaction
- PSBT
- satoshis as micropayment unit
- fee depends on transaction size, not value
- funding transaction
- commitment transaction
- unobserved off-chain events
- payment channel definition
- payment channel step-by-step flow

Ultra-short Week 7 revision

A Bitcoin block header has six fields: version, previous block hash, Merkle root, timestamp, difficulty target, and nonce, and its hash is computed using double SHA-256. Bitcoin's base layer is not ideal for coffee-sized micropayments because blocks take about 10 minutes, throughput is low, fees can be high, and all transactions are public. Bitcoin Script can enforce signatures, multisig, time locks, delay locks, and conditional spending, while Miniscript makes these policies easier to express. In the UTXO model, each input references exactly one previous output, each output has value plus a locking script, and nodes maintain a UTXO

database to validate transactions efficiently. Different children can validly spend different output indices of the same parent, but if two transactions try to spend the same UTXO, they are in a race and only one can win. Standard payments usually need a change output, otherwise leftover value becomes miner fee. Collaborative transactions let multiple parties jointly sign, and PSBTs hold partially signed transactions before final broadcast. Payment channels solve the micropayment problem by using one on-chain funding transaction and many off-chain commitment updates, so only opening and closing touch the blockchain while the intermediate small payments remain fast, cheap, and private.